CATEGORY C SOLUTIONS: Abstraction, Interface & Class vs Interface

STEP SEM-3 CodInClub Powered by BridgeLabz • Complete Implementation Guide

Part A: Coding Problems Solutions

Problem 1: Garden Plot Area Report

Solution Strategy: Use an abstract base class

`Plot`

with an abstract method

`calculateArea()`

. Implement subclasses

`CirclePlot`

,

`RectanglePlot`

, and

`TrianglePlot`

.

```
import java.util.Scanner;

abstract class Plot {
    String owner;
    Plot(String owner) { this.owner = owner; }
    abstract double calculateArea();
    String getOwner() { return owner; }
}

class CirclePlot extends Plot {
    double radius;
    CirclePlot(String owner, double radius) { super(owner); this.radius = radius; }
    double calculateArea() { return Math.PI * radius * radius; }
}

class RectanglePlot extends Plot {
    double length, width;
    RectanglePlot(String owner, double length, double width) { super(owner); this.length =
length; this.width = width; }
    double calculateArea() { return length * width; }
}

class TrianglePlot extends Plot {
    double base, height;
    TrianglePlot(String owner, double base, double height) { super(owner); this.base = base;
this.height = height; }
    double calculateArea() { return 0.5 * base * height; }
}
```

Problem 2: Weekly Staff Pay

Solution Strategy: Create an abstract class

`Staff`

with abstract method

`calculatePay()`

Problem 3: Library Late Fine Counter

Solution Strategy: Abstract class

`LibraryItem`

with fine calculation logic per media type.

```
abstract class LibraryItem {
    String title; int daysLate;
    LibraryItem(String title, int daysLate) { this.title = title; this.daysLate = daysLate; }
    abstract double calculateFine();
    String getTitle() { return title; }
}
class Book extends LibraryItem {
    Book(String title, int days) { super(title, days); }
    double calculateFine() { return daysLate * 2.0; }
}
class DVD extends LibraryItem {
    DVD(String title, int days) { super(title, days); }
    double calculateFine() { return Math.min(daysLate * 5.0, 50.0); }
}
class Magazine extends LibraryItem {
    Magazine(String title, int days) { super(title, days); }
    double calculateFine() { return daysLate * 1.0; }
}
```

Problem 4: Electricity Connection Billing

Solution Strategy: Abstract class

`Connection`

storing units and enforcing

`calculateBill()`

.

```
abstract class Connection {
    double units;
    Connection(double units) { this.units = units; }
    abstract double calculateBill();
}
class HomeConnection extends Connection {
    HomeConnection(double u) { super(u); }
    double calculateBill() { return units <= 100 ? units * 5.0 : (100 * 5.0) + ((units - 100)
* 7.0); }
}
class ShopConnection extends Connection {
    ShopConnection(double u) { super(u); }
    double calculateBill() { return (units * 8.0) + 100.0; }
}
class FactoryConnection extends Connection {
    FactoryConnection(double u) { super(u); }
    double calculateBill() { return Math.max(units * 6.0, 1000.0); }
}
```

Problem 5: Travel Booking with a Common Fee

Solution Strategy: Use template method pattern or abstract base class handling the common fee addition.

```
abstract class Booking {
    double distance;
    static final double BOOKING_FEE = 50.0;
    Booking(double distance) { this.distance = distance; }
    abstract double calculateFare();
    double getTotal() { return calculateFare() + BOOKING_FEE; }
}
```

Part B: Quiz Solutions

Q1. Abstraction definition
Answer: **C** - Hiding how an object performs its operations and showing only what it does.

Q2. Abstract class rule
Answer: **C** - It cannot be instantiated directly.

Q3. Unimplemented abstract method
Answer: **B** - The class must be declared abstract, or the code will not compile.

Q4. Forbidden modifier
Answer: **C** - private (abstract methods must be implemented by subclasses).

Q5. Interface fields
Answer: **C** - public, static, and final.

Q6. Interface implementation count
Answer: **C** - Any number of interfaces.

Q7. Implemented method modifier
Answer: **D** - public (cannot reduce visibility).

Q8. Multiple interfaces vs single class
Answer: **C** - Interfaces carry no instance state, avoiding the 'diamond problem' ambiguity.

Q9. Conflicting default methods
Answer: **D** - A compile-time error occurs until the class overrides the method.

Q10. Abstract class constructors
Answer: **B** - Abstract classes can have constructors that run when a subclass object is created.

Part C: Concept Questions Answers

Question 1: Explain abstraction in your own words with a real-life example.
Abstraction is the principle of exposing essential features while hiding complex implementation details. For example, an ATM machine abstracts away cash verification mechanisms, database encryption, and network socket communications. The user is shown a simple interface (buttons for withdrawal, deposit, balance) while internal banking logic remains completely hidden.

Question 2: Difference between abstraction and encapsulation, and how they work together.
Abstraction focuses on external behavior (what an object does), whereas encapsulation focuses on internal data protection (how data is hidden and safely accessed via getters/setters). Within a single class, encapsulation restricts direct field access using private modifiers, while abstraction exposes public methods that define the high-level contract.

Question 3: Why can an abstract class define a constructor if it cannot be instantiated?
An abstract class cannot be instantiated directly with

```
new
```

, but it can have a constructor to initialize shared state and common fields inherited by subclasses. This constructor is executed via

```
super()
```

when a concrete subclass object is created, ensuring proper setup of parent fields.

Question 4: Why are private, static, and final modifiers forbidden on abstract methods?

```
private
```

is forbidden because abstract methods must be overridden by subclasses.

```
static
```

is forbidden because static methods belong to the class rather than instances and cannot be dynamically dispatched.

```
final
```

is forbidden because final methods cannot be overridden, which contradicts the requirement that subclasses must implement abstract methods.

Question 5: What is the 'diamond problem' and how does Java avoid it?
The 'diamond problem' occurs in multiple inheritance when a class inherits from two parent classes containing methods with identical signatures, creating ambiguity about which one to execute. Java avoids this by disallowing multiple class inheritance while permitting multiple interface implementation, since interfaces contain no instance state and require overriding when default method conflicts arise.